1. Project Overview

1.1 Purpose

The purpose of this project is to design, configure, and test a Raspberry Pi 4 as a functional and practical home router. This project was designed as a hands-on engineering exercise for myself; to build a router that I configured and understood from the ground up instead of relying on service provider systems.

The complete topology uses the Raspberry Pi to exercise core networking functions including DHCP, DNS, NAT, packet forwarding, and firewalling. A dedicated Ethernet switch provides wired connections, while a separate Wi-Fi access point provides wireless connections. I evaluate the wired and wireless capabilities, recording its performance and comparing it with the original Spectrum-provided network. The goal of this comparison is to determine which systems are faster and to understand how network architecture, routing hardware, wireless access, and configuration choices affect real-world performance.

The document reads as a practical lab guide so that the design decisions, configuration process, verification methods, and troubleshooting steps can be understood and reproduced by someone who may not already be familiar with Linux-based routing.

1.2 Final Network Topology

The original network topology was as follows:

Spectrum EN2251 Modem: Brings Internet to the home through the Internet service provider using a Wide Area Network (WAN)

Spectrum SAX1V1K Router: Brings Internet to the home devices using a Local Area Network (LAN)

Wired PC: Connect via Ethernet, this device is Windows operated.

Wireless Devices: Connected via Wi-Fi

Figure 1: Original Network Topology

In comparison, the final network topology separates the wireless network and wired networks through a gigabit Ethernet switch and a Wi-Fi access point. The Spectrum modem remains in both topologies.

Spectrum EN2251

Raspberry Pi 4: Acts as the router, polices traffic including routing, DHCP, DNS, NAT, and firewalls.

GS305 5-Port Gigabit Ethernet Switch: Expands the Raspberry Pi's LAN interface to support multiple devices. Forwards Ethernet frames between devices on the local network. Does not perform routing or function as the default gateway.

TP-Link EAP650 Wi-Fi 6 Access Point: Provides wireless access and bridges Wi-Fi clients onto the wired LAN.

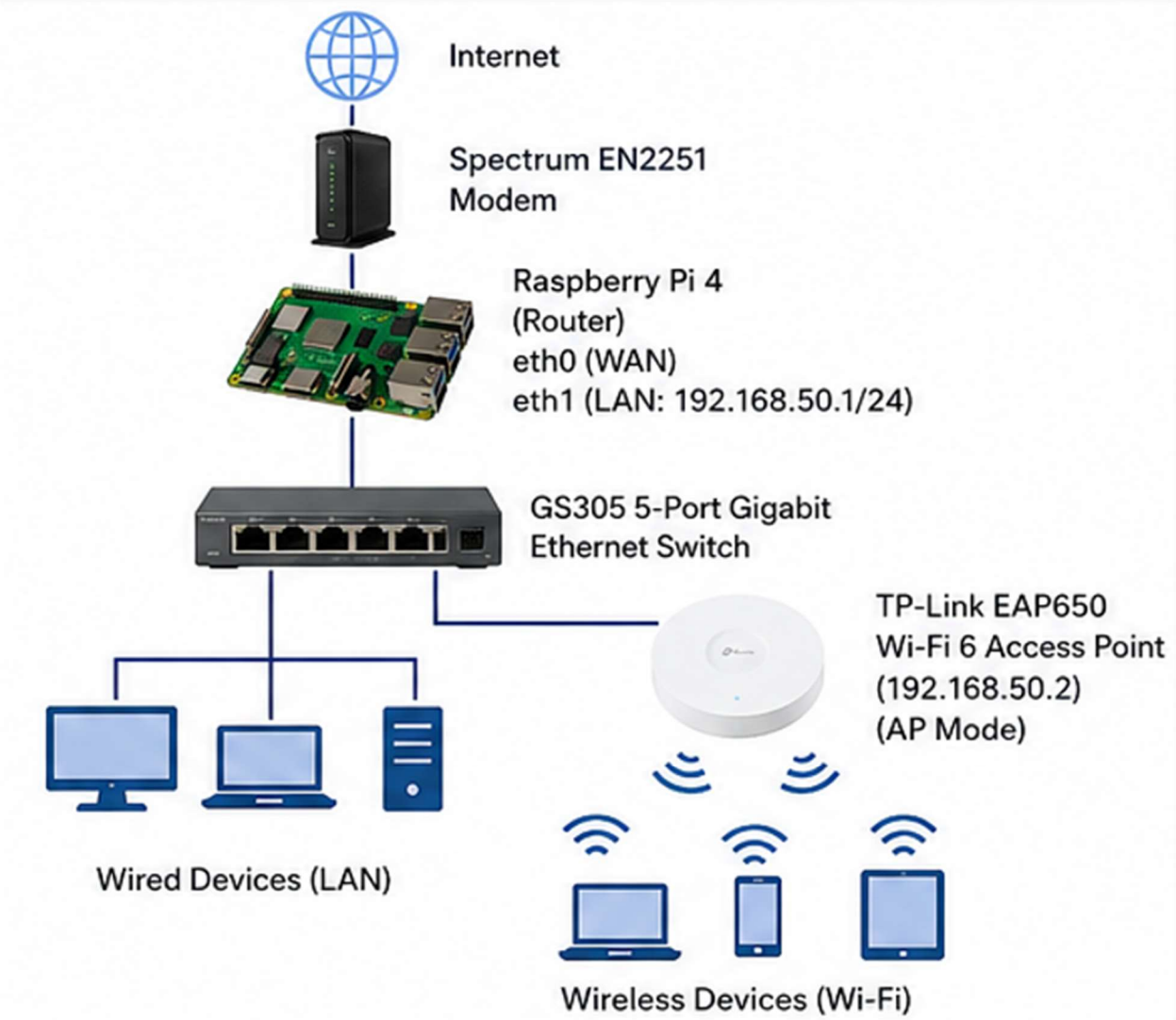

Figure 2: Final Network Topology (Raspberry Pi Router)

The topology creates a modular system wherein each device can be swapped, upgraded, or extended depending on environmental constraints.

1.3 Network Addressing Plan

With the complexity of network addressing, reserve some IP addresses and define others for simplicity. To simplify configuration and troubleshooting, the final topology uses a predefined private addressing plan.

The Raspberry Pi 4 has one built-in Ethernet interface, `eth0`, which is used as the WAN interface. This is automatically assigned an IP address by Spectrum via DHCP. A USB 3.0 Gigabit Ethernet adapter provides the second interface, `eth1`, which is used for private

LAN. The 192.168.50.0/24 subnet was selected for private LAN because it does not overlap with the existing 192.168.1.0/24 network used during initial configuration. The EAP650 management interface is assigned a predictable IP address for administrative access. Assign EAP650 a neighboring address of eth1, 192.168.50.2, reserved by the DHCP. LAN clients will receive dynamic addresses from 192.168.50.100 to 192.168.50.199. The router will use the private subnet 192.168.50.0/24 for all LAN devices. The Raspberry Pi will occupy 192.168.50.1, acting as the default gateway and DNS server.

1.4 Networking Concepts and Terminology

Concept	Definition
Wide Area Network (WAN)	Internet-facing side of the router where data is transmitted over long distances and between networks. This is eth0 connected to the EN2251 modem.
Local Area Network (LAN)	The private network behind the router; sends data to the local IP address. In my system, this is eth1 connected to the Pi to the GS305 switch and the 192.168.50.0/24 network.
IP address	A logical address used to identify a device or interface on an IP network.
Subnet	Defines which addresses belong to the same local network. This is the /24. 192.168.50.0/24 corresponds to the subnet mask 255.255.255.0
Default gateway	The router a client sends packets to when the destination is outside its local subnet. My clients will use 192.168.50.1
Dynamic Host Configuration Protocol (DHCP)	Automatically assigns IP addresses and other essential network settings. In my design, dnsmasq performs these processes
Domain Name System (DNS)	Translates names such as example.com into IP addresses that computers can route to. The Raspberry Pi acts as the clients' DNS forwarder/cache
Network Address Translation (NAT)	Translates traffic from private LAN addresses to the WAN address used on Internet-facing side of the Pi
Packet	A formatted unit of data transmitted across a network. Routers examine packet addressing information to determine where the packet should go
Packet forwarding/routing	The process of accepting a packet on one interface and sending it out to another. In my router, this is eth1 to eth0

Firewall	The rules controlling which network traffic is permitted or rejected. My design uses nftables to define this
Medium Access Control (MAC) Address	A link-layer identifier associated with a network interface. This is particularly relevant in reserving `192.168.50.2` for the EAP650
Network Interface	A connection through which the Pi communicates. My final system, `eth0` will be my WAN, `eth1` will be my LAN, and `wlan0` will be disconnected.
Ethernet switch	Connects multiple wired devices on the same LAN. My GS305 expands the Pi's Lan from one Ethernet connection to five Ethernet connections.
Wireless access point	Connects Wi-fi clients onto the wired LAN. The EAP650 provides Wi-Fi while the Raspberry Pi remains responsible for networking configurations.
SSID	The Wi-Fi network name broadcast by the access point

Table 1: Network Concepts and Terminology

2. Materials and Prerequisites

Hardware: 5-port Gigabit Ethernet Switch, Wi-Fi 6 Access Point, USB 3.0 to Gigabit Ethernet Adapter, Raspberry Pi 4 Model B with Official Case and Power Adapter, Raspberry Pi 4 Cooling Fan, Four Cat6 Ethernet Cables (50 ft, 7 ft, 3 ft, 3 ft), 128 GB microSD Card with Adapter, Compatible computer for SSH

PRIVACY WARNING: REDACT MAC ADDRESSES, PUBLIC IPs, CREDENTIALS, AND PERSONALLY IDENTIFYING NETWORK NAMES WHEN PUBLISHING

3. Raspberry Pi Preparation

Assemble Raspberry Pi 4: Install the Raspberry Pi in its case and attach the cooling fan and heatsink. The active cooling will help maintain a stable operating temperature and efficiency, reducing the likelihood of thermal throttling.

Flash the microSD: Install and launch the Raspberry Pi Imager application with the microSD connected to the computer. Select the Raspberry Pi 4 as the target device and install the 64-bit Raspberry Pi OS. Provide a hostname, username, and password of your choosing, then enable SSH and password authentication. Writing the image will erase any data on the SD card.

Warning: Writing the operating system image erases all existing data on the selected microSD card. Verify that the selected storage device is correct before continuing.

Connect to the Raspberry Pi: Insert the microSD with the Raspberry Pi OS after properly ejecting the SD card. Power the Raspberry Pi with the power adapter. Open PowerShell or terminal or command-line application on a computer.

```
ping <hostname>.local
```

The command ping sends an ICMP Echo Request packet to the specified hostname. Successful replies verify that the computer can resolve the Pi's hostname and communicate with it over the local network.

```
ssh <username>@<hostname>.local
```

Secure Shell (SSH) provides command-line access to the Raspberry Pi from another computer on the network. For the first connection, SSH will prompt the user with a yes/no/fingerprint question to determine whether the host key should be trusted. Confirm the prompt and enter the password given during the Raspberry Pi Imager setup.

Confirm the OS by running the following command

```
cat /etc/os-release | grep PRETTY_NAME
```

In this instance, `PRETTY_NAME = "Debian GNU/Linux 13 (Trixie)"` which means the Raspberry Pi OS is Debian-based.

Update the Raspberry Pi OS:

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

The first command refreshes the system's package index, so the Pi knows which software versions are available. The second command downloads and installs the available package and dependency updates. The third command restarts the Pi so system-level updates can take effect. `sudo` executes the command with administrator privileges.

With the Raspberry Pi updated, begin identifying the network interfaces that will be used to configure it as a router.

```
ip link show
```

This command displays the Pi's available network interfaces and their current link states. The relevant interfaces are as follows.

lo is the loopback interface used internally by the operating system.

eth0 is the Raspberry Pi's built-in Ethernet port which will eventually be used as the WAN port.

wlan0 is the Pi's built-in Wi-Fi. While configuring, this provides the temporary network and SSH access through the existing Spectrum router.

When `eth0` displays `NO-CARRIER` and `state DOWN`, this means there is no physical Ethernet link present. This is expected before the WAN cable is connected.

Values formatted as `aa:bb:cc:dd:ee:ff` are MAC addresses. A MAC address identifies a network interface at the local link layer and will be useful later when identifying devices on a local network and creating DHCP reservations.

4. LAN Configuration

Plug in the USB 3.0 to Gigabit Ethernet Adapter into one of the Pi's blue USB 3.0 ports. Wait 10 seconds after connection and run the following commands in the Raspberry Pi terminal, either locally or through SSH.

```
lsusb
nmcli device status
ip -br link
```

The command `lsusb` lists the USB devices and confirms whether Linux (Raspberry Pi OS uses Debian-based Linux OS) detects the adapter as USB hardware. The second command `nmcli device status` gives a list of all network interfaces and their current states. The command `ip -br link` gives a shorter version of `ip link show`

Implementation Note: The USB Ethernet adapter appears as eth1 in this build. Interface names can vary between systems. Verify the interface name using `nmcli device status` or `ip -br link` before continuing and substitute the appropriate interface name if necessary.

The output should contain an entry identifying the RTL8153 Gigabit Ethernet adapter. The USB bus and device numbers may vary. RTL8153 Gigabit Ethernet Adapter confirms that the Pi does detect the connection. The following command confirms that Raspberry Pi is connected to the Wi-Fi via `wlan0`.

Implementation Note: `eth1` initially shows as `connecting` because the automatically generated NetworkManager profile was attempting to obtain an address through DHCP. As the GS305 is an unmanaged switch and does not provide DHCP, no address is offered.

```
ip -4 -br address show wlan0
```

The result will be the existing service provider's router network. The existing network address shown on `wlan0` should not overlap with the private LAN defined in the Network Addressing Plan. In this build, the existing Spectrum network uses `192.168.1.0/24`, so the Raspberry Pi LAN is assigned `192.168.50.0/24`, with `192.168.50.1` assigned to `eth1`. With the LAN subnet selected, configure `eth1` with the planned static address

```
sudo nmcli connection add type ethernet ifname eth1 con-name pi-lan
ipv4.method manual ipv4.addresses 192.168.50.1/24 ipv4.never-default
yes ipv6.method disabled connection.autoconnect yes

sudo nmcli connection up pi-lan

ip -4 -br address show eth1
```

The first command creates a NetworkManager Ethernet profile named pi-lan and applies it to `eth1`. The profile assigns the static address `192.168.50.1/24` instead of requesting an address through DHCP. The `ipv4.never-default yes` option prevents the LAN interface from becoming the Pi's default Internet path, while `ipv6.method disabled` temporarily disables IPv6 on this test LAN to simplify the initial configuration. The profile is also configured to automatically reconnect when the Pi boots. NetworkManager uses connection profiles to hold settings such as static IP addresses. `nmcli` is its command-line configuration tool. The second command activates the connection. The third command verifies that the `eth1` connection has the provided IP address.

Run the following commands to confirm the new LAN configuration:

```
nmcli device status

ip route
```

The expected result is that `eth1` should show connected using the `pi-lan` profile while `wlan0` remains the temporary Internet and SSH connection. The routing table should contain the `192.168.50.0/24` LAN route without replacing the existing default route through `wlan0`. This confirms that the Pi now has a dedicated private LAN while retaining its temporary connection to the existing network.

5. DHCP and DNS Configuration

Install `dnsmasq`, a software tool designed to provide DNS forwarding and caching, DHCP, and TFTP services for small networks.

```
sudo apt update
```

```
sudo apt install dnsmasq -y
sudo systemctl stop dnsmasq
sudo apt install bind9-dnsutils
```

Temporarily stop `dnsmasq` while its configuration file is being created. The command `systemctl` is used to manage background system services running on the system. Install `bind9-dnsutils` to provide DNS diagnostic utilities such as `nslookup` and `dig`, which will be used later to verify DNS operation.

```
sudo tee /etc/dnsmasq.d/pi-router.conf > /dev/null << 'EOF'
# Listen only on the Raspberry Pi private LAN
interface=eth1
listen-address=192.168.50.1
bind-dynamic

# DHCP address pool for LAN clients
dhcp-range=192.168.50.100,192.168.50.199,255.255.255.0,12h

# Tell clients that the Pi is their router and DNS server
dhcp-option=option:router,192.168.50.1
dhcp-option=option:dns-server,192.168.50.1

# Local DNS configuration
domain=lan
local=/lan/
expand-hosts
domain-needed
bogus-priv
cache-size=1000

# Record DHCP activity in the system log
log-dhcp
EOF
```

The command writes the configuration into `/etc/dnsmasq.d/pi-router.conf`. The interface and listen-address settings restrict DNS service to the Raspberry Pi's private LAN instead of offering it on every network interface.

- `dhcp-range` setting activates the DHCP server for that network.
- `interface=eth1` provides DHCP and DNS through the USB Ethernet LAN interface.
- `listen-address=192.168.50.1` listens for DNS requests at the Raspberry Pi's LAN address.
- `bind-dynamic` binds the service to the selected interface address while allowing it to handle interface changes cleanly.
- `dhcp-range=192.168.50.100,192.168.50.199,255.255.255.0,12h` gives clients addresses from `192.168.50.100` through `192.168.50.199` and each lease lasts 12 hours before renewal.
- `dhcp-option=option:router,192.168.50.1` tells the clients that the Raspberry Pi is their default gateway.
- `dhcp-option=option:dns-server,192.168.50.1` tells the clients to send DNS questions to the Raspberry Pi.
- `domain=lan` defines lan as the local domain for this network
- `local=/lan/` keeps the requests for the `.lan` domain local instead of forwarding them upstream
- `expand-hosts` allows for simple local hostnames to be expanded using the local domain
- `domain-needed` prevents incomplete hostnames without a domain from being forwarded to upstream DNS servers
- `bogus-priv` prevents reverse DNS queries for private IP addresses from being forwarded to public DNS servers
- `log-dhcp` records DHCP activity for troubleshooting and verification
- `cache-size=1000` allows the Pi to retain recent DNS answers, reducing repeated upstream lookups.

Verify that the configuration file was written correctly

```
cat /etc/dnsmasq.d/pi-router.conf
```

It should display the configuration entered above. Test the configuration syntax.

```
sudo dnsmasq --test
```

The desired result will be `dnsmasq: syntax check OK`

Re-enable `dnsmasq` and configure it to start automatically at boot.

```
sudo systemctl enable --now dnsmasq
```

Check its status via

```
sudo systemctl status dnsmasq --no-pager
```

Or its shorten version

```
systemctl is-active dnsmasq
```

To return just its active status

```
sudo ss -luntp | grep dnsmasq
```

This lists the network ports being used by `dnsmasq`. Port 53 is the DNS server and port 67 is the DHCP server. Standard Internet protocols. A newly connected DHCP client sends requests from the User Datagram Protocol (UDP) port 68 to the DHCP server on UDP port 67. The Raspberry Pi, acting as the DHCP server responds from port 67 to the client on port 68.

Test the LAN connection by pinging the Pi

```
ping 192.168.50.1
```

A successful ping confirms that the wired client can reach the Raspberry Pi through `eth1`. Test the DNS via `bind9-dnsutils` command

```
nslookup example.com 192.168.50.1
```

This will explicitly ask the Pi to resolve `example.com`, confirming that the Pi can identify the IP address of a named domain. Viewing the DHCP leases on the Pi can be done by running the command

```
cat /var/lib/misc/dnsmasq.leases
```

This will produce the lease expiration time, client MAC address, the assigned IP address, and client hostname

To watch the DHCP activity live, open a second terminal and, through the Raspberry Pi, run

```
sudo journalctl -u dnsmasq -f
```

Then disconnect and reconnect the client's Ethernet cable. This should provide messages for `DHCPDISCOVER`, `DHCPOFFER`, `DHCPREQUEST`, and `DHCPACK`.

`DHCPDISCOVER` - client asks whether a DHCP server exists

`DHCPOFFER` - server offers an address

`DHCPREQUEST` - client requests the offered address

DHCPACK - server approves the lease.

Implementation Note: In this build, only DHCPREQUEST and DHCPACK were observed because the client already had knowledge of its previous lease. A new client or expired lease may show the complete four-message exchange.

You can stop the live log with Ctrl + C. This only stops the live log and not dnsmasq. This can be confirmed by

```
systemctl is-active dnsmasq
```

Disable Wi-Fi on the client computer so that all traffic must pass through the wired Raspberry Pi LAN. In PowerShell, run

```
ipconfig /all
```

Verify that the Ethernet interface received an address from the 192.168.50.100 through 192.168.50.199 DHCP pool, with 192.168.50.1 listed as both the default gateway and DNS server.

Next, ping 192.168.50.1. Successful replies confirm that the packets can travel through the client Ethernet interface, GS305 switch, USB Ethernet adapter, and eth1.

Run

```
nslookup example.com 192.168.50.1
```

A successful response confirms that the wired client can send DNS requests to Raspberry Pi and receive resolved addresses.

Finally, ping 1.1.1.1 from the wired client to test Internet forwarding. At this stage, this is expected to fail. While the Raspberry Pi itself has Internet access through wlan0, IPv4 forwarding and NAT have not yet been configured. The Pi can therefore provide DHCP and DNS services to the client but cannot yet route the client's Internet traffic between eth1 and wlan0. Packet forwarding and NAT are configured in the following section.

6. Routing, NAT, and Firewall Configurations

Enable packet forwarding in a temporary manner to confirm its functionality before making persistent change. This section begins work on NAT masquerading, so packet sources are translated to the Pi's Wi-Fi address. With the previous conditions remaining unchanged, run the following command through the Raspberry Pi to enable IPv4 temporarily

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
sysctl net.ipv4.ip_forward
```

The results are as expected, `net.ipv4.ip_forward = 1`. This setting will only last until the Pi reboots. Begin with NAT configuration by installing `nftables`.

```
sudo apt install -y nftables
```

Before adding custom `nftables` rules, check whether UFW is installed and active.

```
sudo ufw status
```

If UFW is not installed, the command may return `command not found`. If it is installed and active, review the existing firewall configuration before continuing to avoid conflicting rules.

```
sudo nft list ruleset
```

Inspect the existing `nftables` ruleset. In a clean installation, it may be empty. If rules are present, review them before loading the configuration below:

```
sudo nft -f - <<'EOF'
table ip pi_router {
    chain forward {
        type filter hook forward priority 0; policy drop;

        iifname "eth1" oifname "wlan0" \
            ct state { new, established, related } counter accept

        iifname "wlan0" oifname "eth1" \
            ct state { established, related } counter accept
    }

    chain postrouting {
        type nat hook postrouting priority 100; policy accept;

        oifname "wlan0" ip saddr 192.168.50.0/24 \
            counter masquerade
    }
}
EOF
```

The rule permits new connections from private LAN to the temporary Wi-Fi uplink.

```
eth1 -> wlan0
```

The second rule permits only established or related response traffic back from `wlan0` to `eth1`.

This permits LAN clients to initiate outbound connections while allowing only established or related return traffic from the temporary WAN side.

- `policy drop` rejects forwarded traffic unless a rule explicitly permits it
- `ct state` uses connection tracking to distinguish new connections from traffic belonging to existing connections
- `established` and `related` allow legitimate response traffic to return to LAN clients
- `counter` records the number of packets and bytes matching the rule
- `masquerade` performs source NAT using the address of the outgoing WAN interface

New unsolicited connections arriving through `wlan0` are not forwarded into the private LAN. The `masquerade` rule replaces the source addresses of `192.168.50.0/24` clients with the Pi's `wlan0` address. In this temporary setup, the existing Spectrum router performs NAT again before the traffic reaches the Internet. This creates a double-NAT configuration, which is acceptable for testing, but is removed during the final WAN cutover.

Confirm that the rules are loaded

```
sudo nft list table ip pi_router
```

In PowerShell, ping the Pi to confirm LAN connectivity, then ping `1.1.1.1` to test Internet routing without relying on DNS. Next, run:

```
ping example.com
```

Successful replies confirm that DNS resolution is functioning in addition to Internet routing. Finally, test HTTPS traffic with:

```
curl.exe -I https://example.com
```

This HTTPS test receives a successful reply of `HTTP/1.1 200 OK` confirming that ordinary Internet traffic is passing through the Pi. Watch the path of `1.1.1.1` with:

```
tracert 1.1.1.1
```

Implementation Note: In this build, tracert reached the destination in 14 hops, with 4 intermediate hops not returning responses. The route also demonstrated the temporary two-router topology created by the Raspberry Pi and existing Spectrum router.

Returning to the Pi, watch the nftables packet counter live for future testing

```
watch -n 2 'sudo nft list table ip pi_router'
```

Press Ctrl + C to stop the live display.

Once the temporary routing configuration has been validated, make IPv4 forwarding and the nftables rules persistent across reboots.

```
sudo tee /etc/sysctl.d/99-pi-router.conf > /dev/null <<'EOF'
net.ipv4.ip_forward=1
EOF
```

Load it immediately

```
sudo sysctl --system
```

Verify

```
sysctl net.ipv4.ip_forward
```

Then save the current `nftables` rules

```
sudo sh -c '{
echo "#!/usr/sbin/nft -f"
echo
echo "flush ruleset"
nft list ruleset
} > /etc/nftables.conf'
```

This command writes the complete currently loaded `nftables` ruleset to `/etc/nftables.conf`. The `flush ruleset` line clears previously loaded rules before the saved configuration is restored, preventing duplicate tables or rules during startup. Display the file.

```
sudo cat /etc/nftables.conf
```

Then test the saved file before enabling it.

```
sudo nft --check --file /etc/nftables.conf
```

If it returns without issue, enable nftables at startup and confirm its activity.

```
sudo systemctl enable nftables
systemctl is-enabled nftables
sudo systemctl restart nftables
systemctl is-active nftables
```

Restarting nftables reloads `/etc/nftables.conf`, providing an opportunity to verify that the saved configuration works independently of the temporary rules created earlier. From the wired Windows client, repeat the Internet-routing and HTTPS test. Successful results confirm that the saved `nftables` configuration is functional. Confirm that the other services are still working as intended.

```
systemctl is-enabled dnsmasq
systemctl is-active dnsmasq
nmcli -g connection.autoconnect connection show pi-lan
```

Initiate the reboot test while the following conditions remain true. Pi Wi-Fi remains configured, Ethernet connections remain connected, and `eth0` remains unplugged.

```
sudo reboot
```

Through the Raspberry Pi, verify the interface addresses, IPv4 forwarding state, service status, and `nftables` rules.

```
ip route
ip -4 -br address
sysctl net.ipv4.ip_forward
systemctl is-active dnsmasq nftables
sudo nft list table ip pi_router
```

After the Raspberry Pi reboots, repeat the client-side network validation procedure in Appendix B. Successful LAN, Internet, DNS, and HTTPS tests confirm that the router configuration persists across a reboot.

7. Wireless Access Point

The EAP650 provides the wireless access portion of the router system. With the persistent routing configuration verified in the previous section, connect and power the EAP650 access point. Connect its Ethernet interface to the GS305 switch. The access point should receive an address from the Raspberry Pi's DHCP server.

```
cat /var/lib/misc/dnsmasq.leases
```

This command gives a lease like

```
<expiration> aa:bb:cc:dd:ee:ff 192.168.50.126 EAP650 <client-id>
```

Implementation Note: During this build, the EAP650 also presented a DHCP client identifier beginning with `01:`. The leading `01` identifies the hardware type as Ethernet and is not part of the six-byte MAC address. This distinction became important when creating the DHCP reservation below.

Begin to configure the EAP650 via the Omada app in standalone mode given the single access point setup for this topology. Connect to the EAP650's default Wi-Fi name printed on its label, open Standalone Mode in the app, select the EAP650, create new administrator credentials, and create the new Wi-Fi network name and password. Configure a recognizable SSID and a secure wireless password. This can be changed at any point without altering the Raspberry Pi routing configuration. Give the Wi-Fi Access Point a recognizable and predictable IP address for management purposes, reserving the IP address `192.168.50.2`. To create the reservation, use the EAP650's MAC address identified in the DHCP lease.

```
sudo nano /etc/dnsmasq.d/pi-router.conf
```

Add the following line to the bottom of the dnsmasq configuration:

```
dhcp-host=aa:bb:cc:dd:ee:ff,id:*,192.168.50.2,eap650,infinite
```

Save in Nano using Ctrl + O, Enter, Ctrl + X.

The reservation assigns `192.168.50.2` to the EAP650 management interface. The `id:*` option instructs `dnsmasq` to match the reservation using the hardware address rather than requiring a particular DHCP client identifier.

Check that the syntax is correct using

```
sudo dnsmasq --test
```

```
sudo systemctl restart dnsmasq
```

Reboot the EAP from the Omada app and verify its IP address from the app and the Pi by checking the `dnsmasq` leases.

```
cat /var/lib/misc/dnsmasq.leases
```

This should allow wireless connections and Internet services to be accessed. Confirm this using a separate device and connect to the EAP650 Wi-Fi. In PowerShell, check the current IP network configuration and ensure it is receiving the appropriate IP addresses for the Raspberry Pi. After the client receives a `192.168.50.x` address, repeat the client-side validation procedure in Appendix B to verify DHCP, DNS, LAN connectivity, and Internet access through EAP650. Confirm the wireless DHCP activity on the laptop through the Pi using the following command

```
sudo journalctl -u dnsmasq -f
```

On the wireless client, disconnect from and reconnect to the EAP650 SSID while monitoring the `dnsmasq` log. DHCP activity such as `DHCPDISCOVER`, `DHCPOFFER`, `DHCPREQUEST`, and `DHCPACK` may appear. As an optional step, update the EAP through the Omada app

8. Final WAN Cutover

Confirm the WAN interface configuration: Before modifying the firewall, inspect the available NetworkManager profiles and confirm that `eth0` has a profile configured for automatic addressing

```
nmcli connection show
```

```
nmcli device status
```

Raspberry Pi OS may already provide an automatically generated profile for eth0. If a suitable automatic Ethernet profile is present, no additional WAN profile is required

Implementation Note: A dedicated `pi-wan` NetworkManager profile was initially created for `eth0`. During final testing, Raspberry Pi OS automatically activated its existing netplan-`eth0` profile, which successfully obtained WAN configuration from the Internet Service Provider (ISP). The additional profile was therefore not required for normal operation.

Prepare the firewall for the wired WAN: The temporary nftables configuration forwards LAN traffic through `wlan0`. Before replacing the Spectrum router, create a backup of the working rules and modify the saved configuration to use `eth0` as the WAN interface.

```
sudo cp /etc/nftables.conf /etc/nftables.conf.wifi-backup
```

Check relevant lines to see the current configuration routing

```
sudo grep -nE 'wlan0|eth0|eth1|masquerade' /etc/nftables.conf
```

Only the saved configuration is modified at this stage. The currently active firewall continues using `wlan0`, preserving Internet access while the final WAN connection is being prepared

```
sudo sed -i 's/"wlan0"/"eth0"/g' /etc/nftables.conf
```

This replaces references to the temporary WAN interface, `wlan0`, with the final WAN interface, `eth0`, in the saved nftables configuration. The currently active rules remain unchanged until `nftables` is restarted after the physical cutover. Check only the relevant

lines in the current configuration routing to confirm the routing changes. Confirm that the edited file is valid

```
sudo nft --check --file /etc/nftables.conf
```

From the PC, ping the Raspberry Pi from cmd and test ssh directly over Ethernet via

```
ssh <username>@192.168.50.1
```

Verify that the Raspberry Pi can be administered through its LAN address before disconnecting the temporary Wi-Fi path. After the Spectrum router is removed, `wlan0` will no longer provide the existing SSH connection.

Warning: Confirm SSH access through `192.168.50.1` before disconnecting the existing router. Otherwise, remote access to the Raspberry Pi may be lost during the cutover.

Shutdown the Raspberry Pi using:

```
sudo poweroff
```

Power off the Spectrum modem and router before changing Ethernet connections. Disconnect the Spectrum router from the modem and connect the modem's Ethernet port directly to the Raspberry Pi's built-in `eth0` interface.

The final topology now follows figure 2.

Power on the modem first and wait until it has fully synchronized with the ISP. Then power on the Raspberry Pi.

Inspect the `eth0` via

```
ip -4 -br address show eth0
ip route
```

Privacy Note: The IPv4 address assigned to `eth0` may be a public Internet address. Redact it before publishing screenshots or terminal output

`eth0` should receive an address automatically from the ISP, and the default route should now use `eth0`.

Activate the prepared firewall

```
sudo systemctl restart nftables
sudo nft list table ip pi_router
sysctl net.ipv4.ip_forward
```

Do two quick tests to confirm the pathing from the Raspberry Pi to the Internet

```
ping -c 4 1.1.1.1
ping -c 4 example.com
```

The first test verifies Internet connectivity without DNS resolution. The second verifies DNS resolution in addition to Internet connectivity

In cmd, test the wired PC with the validation procedure in appendix B to verify DHCP, DNS, LAN connectivity, and Internet access with a `tracert 1.1.1.1`. Connect a wireless client to the EAP650 and repeat Appendix B client-side validation procedure. This confirms that both wired and wireless clients are now reaching the Internet through the Raspberry Pi's `eth0` WAN interface.

Once wired and wireless operation have been verified through `eth0`, disable automatic connection to the previous Wi-Fi network and disconnect `wlan0`

```
nmcli connection show
sudo nmcli connection modify <old-wifi-profile> connection.autoconnect
no
sudo nmcli connection down <old-wifi-profile>
nmcli device status
ip route
```

`eth0` should remain connected as the WAN interface, `eth1` should remain connected using the `pi-lan` profile, and `wlan0` should be disconnected. The default route should use `eth0`.

This completes the replacement of the Spectrum router with the Raspberry Pi routing system, providing both wired connectivity through the GS305 switch and wireless connectivity through the EAP650 access point.

9. Performance Testing

9.1 Test Methodology

Performance testing was conducted in a way to accurately compare the completed Raspberry Pi routing system against the legacy Spectrum-provided router. Testing compared the Raspberry Pi and Spectrum routing systems at three locations while keeping the router position constant between configurations. Measurements were collected using Speedtest on the same server to accurately measure download throughput, upload throughput, idle latency, download-loaded latency, and upload-loaded latency. For the Raspberry Pi routing system, temperature was also recorded.

Wired and wireless tests were performed in an alternating fashion to reduce time variability between wired and wireless with ten paired measurements for each location. This minimized the influence of changing ISP or Speedtest server conditions. The wireless client was the same across all wireless tests, in these tests, a laptop. The wireless locations were taken from increasing distance from the Wi-Fi access point

- Ideal: approximately 3 ft from the wireless access point
- Living Room: approximately 16 ft away with line of sight
- Bedroom: approximately 20 ft away with the bedroom door closed and no direct line of sight

These locations were used to evaluate the wireless capabilities of the different routing systems. The wired measurements functioned as a baseline through which the wireless systems could be compared to in terms of peak performance at that specific period. This is recorded as the wireless throughput retention rate, defined as the wireless download throughput divided by the corresponding wired download throughput. This metric will indicate how much of the Internet connection remained available from Ethernet over to the wireless client.

9.2 Wired Routing Performance

The wired tests isolate the performance of routing from the Wi-Fi access point performance. The Raspberry Pi produced download speed average ranging from 544 and 548 Mbps across the three test locations. The Spectrum router produced averages around 537 and 544 Mbps. Both systems performed above the service rate of 500 Mbps. In testing, Raspberry Pi showed slightly greater download throughput averages.

Test Location	Raspberry Pi Wired Download Throughput	Spectrum Wired Download Throughput
Ideal	544.3 Mbps	537.1 Mbps
Living Room	546.6 Mbps	538.2 Mbps
Bedroom	548.7 Mbps	543.7 Mbps

Table 2: Wired Routing Download Speed Results

The wired tests showed greater differences in latency. The Raspberry Pi showed wired idle latency around 9.7 to 10 ms. In contrast, the Spectrum wired idle latency ranged around 10.7 to 11.4 ms. A similar trend is observed in download-loaded latency where the

Raspberry Pi recorded download-loaded latency around 42.5 to 47.3 ms in comparison to Spectrum's 62.1 to 69.8 ms download-loaded latency. The largest difference occurred between the Raspberry Pi and Spectrum wired upload-loaded latency tests. In these recordings, the Raspberry Pi recorded timings that averaged 7.9 ms across all tests, whereas Spectrum's performance was around 188.3 to 202.5 ms. The wired download throughput had relatively small variability between both systems

The upload throughput recordings across all tests remained around 23 Mbps for both routing systems, wired and wireless. As neither configuration produced a meaningful change in upload throughput, the availability of upstream bandwidth appears to be limited by the ISP rather than either router's capabilities.

These results suggest that the Raspberry Pi could perform the primary responsibilities of a wired router without the introduction of meaningful throughput bottleneck.

9.3 Wireless Performance

The wireless testing showed larger differences between each routing system in contrast to the wired testing. Results include raw download speeds as well as the retention rate relative to the wired throughput.

$$\frac{average\ wireless\ download\ speed}{average\ wired\ download\ speed} X\ 100\% = wireless\ retention\ rate$$

Equation 1: Wireless Retention Rate Calculation

Location	Raspberry Pi + EAP650	Spectrum Router	Raspberry Pi Wireless Retention	Spectrum Wireless Retention
Ideal	535.9 Mbps	534.7 Mbps	98.5%	99.5%
Living Room	465.5 Mbps	488.1 Mbps	85.2%	90.7%
Bedroom	330.4 Mbps	425.2 Mbps	60.2%	78.2%

Table 3: Wireless Routing Download Speed and Retention Rate Results

In evaluating the ideal conditions for wireless systems, both setups showed near identical performances where nearly all the available throughput was delivered with respect to their respective wired connections. The living room tests begin performance over a distance, recording degradation of wireless throughput. The Raspberry Pi performs at 465.5 Mbps, while the Spectrum router performs slightly greater at 488.1 Mbps. The largest difference between the two systems is presented in the most obstructive location, the bedroom. In

this third location, the EAP650 averaged 330.4 Mbps while the Spectrum router averaged 425.2 Mbps, a substantial difference of 95 Mbps. With relation to the wireless retention rate, this figure presents itself more accurately as a fall to 60.2% for the EAP650 from its wired throughput. In contrast, the Spectrum router retained approximately 78.2 % of its wired throughput. The standard deviations indicate that the Spectrum router was more consistent than the Raspberry Pi/EAP650 in the ideal tests; Spectrum's 6.95 Mbps versus Raspberry Pi/EAP650's 17.07 Mbps. The Bedroom and Living Room tests, the Spectrum presented higher raw averages than the Raspberry Pi/EAP650, but the standard deviations indicate that the Raspberry Pi produced more consistent download-throughput measurements. Living Room had results of 28.73 Mbps and 53.05 Mbps, Raspberry Pi/EAP650 and Spectrum, respectively. Bedroom had results of 15.25 Mbps and 79.6 Mbps for Raspberry Pi/EAP650 and Spectrum, respectively.

Measurements recorded from the latency tests provided interesting results. The idle latency timings from the Raspberry Pi/EAP650 system resulted in 23.7 to 27.7 ms. The Spectrum router had timings from 26.7 to 41.4 ms with a high standard deviation of 23.84 ms in comparison to the EAP650's high 7.6 ms standard deviation in idle latency. The download latency had the largest difference in latency tests. The Raspberry Pi/EAP650 had download latency range of 66.8 to 159.8 ms with the standard deviation being 18.4 ms. In raw timings, the Spectrum router performed faster in download latency with its range of 65.6 to 95.0 ms, yet more inconsistent with a high deviation of 20.72 ms. The upload latency between both systems provided comparable results and similar deviations. Raspberry Pi/EAP650: 204.5 to 207 ms, a high deviation of 12.81 ms. Spectrum: 199.9 to 209 ms, a high deviation of 14.46 ms.

The results from the wireless tests do not indicate that the Raspberry Pi is at fault for the decreased retention rate in wireless throughput as its wired performance remained consistent as the locations changed. The reduction can be attributed to the wireless portion of the topology rather than the Raspberry Pi routing layer, with the EAP650 and its RF environment being the primary variables. The Spectrum router's measurements suggest a similar idea to the router's integrated wireless system that is responsible for the reduction of retention.

These tests present a clear separation of wired and wireless capabilities. The Raspberry Pi performed strongly in the wired tests, but the Spectrum router proved superior in raw wireless throughput as distance/obstruction increased.

9.4 Raspberry Pi Thermal Performance

The Raspberry Pi had consistent thermal performance throughout the testing with the following average temperatures:

Test Location	Average Wired Temperature	Average Wireless Temperature
Ideal	44.02 ℃	42.95 ℃
Living Room	43.8 ℃	43.1 ℃
Bedroom	44.55 ℃	44.19 ℃

Table 4: Raspberry Pi Thermal Performance, Wired and Wireless

In sustained Speedtest traffic, the temperatures remained consistent and without substantial differences in operating temperatures. The wireless temperatures consistently had lower operating temperatures than wired, but nothing substantial.

Comparison with Spectrum router was not performed because equivalent internal temperature telemetry was not available from the Spectrum router.

9.5 Testing Limitations

The results from this project were performed in a specific home environment. Wireless performance relies heavily on environmental factors including access point placement, building materials, interference, client hardware, antenna design, and radio configuration. The ideal wireless test was performed in such a way to limit these factors as much as possible and represents favorable RF conditions rather than typical household usage.

Internet-based Speedtest measurements can also vary because of ISP conditions and test-server load. This is previously explained in testing methodology where tests between wired and wireless were alternated. However, this does not eliminate variability in conditions. The wired measurements therefore provide an important baseline for determining whether changes in wireless throughput occurred while the available Internet connection remained stable.

Conclusions and Lessons Learned

The Raspberry Pi successfully replaced the Spectrum router as the primary routing platform without introducing a wired throughput bottleneck. It matched or slightly exceeded the Spectrum router's wired throughput while producing lower wired loaded latency in recorded tests. While the maximum difference was only 10 Mbps in download speeds, the Raspberry Pi proved more consistent overall. This trend continues with the wireless setups, even though the EAP650 overall performed worse in raw download

speeds. Another point contention is the latency tests. Wireless results were more mixed where the EAP650 matched Spectrum in ideal conditions, but retention declined as distance and obstruction increased. However, the EAP650 produced lower download-throughput variability than Spectrum in living room and bedroom tests. An interesting point about Spectrum's upload latency tests is that the results were nearly identical in wired versus wireless measurements. A small note about the upload latency for Spectrum is that the ideal tests show the best raw timings, but the most inconsistent. At greater distances and obstruction, the Spectrum wireless system maintained higher raw throughput, while the EAP650 produced lower download-throughput variability. The Raspberry Pi/EAP650 provided the stronger wired routing performance in these tests, while the wireless comparison revealed a tradeoff between the EAP650's lower variability at more difficult locations and the Spectrum router's higher retained throughput.

This project provided meaningful experiences in Internet traffic, routing system, and a deeper understanding of wireless Wi-Fi systems.

Appendix

Appendix A – Command Reference

This appendix summarizes the primary Raspberry Pi/Linux commands used throughout this project. Commands are grouped by function so they can be referenced without repeating their full explanations throughout the main procedure.

A.1 System Administration

Command	Purpose	Typical Use	Notes
`sudo`	Run a command with administrator privileges	System configuration	Applies only to the command that follows
`sudo apt update`	Refresh available package information	Software setup	Does not install upgrades by itself
`sudo apt full-upgrade -y`	Install available system/package updates	Initial Pi setup	`-y` automatically confirms prompts
`sudo systemctl`	Control and inspect system services	`dnsmasq`, `nftables`	Common actions: `start`, `stop`, `restart`, `enable`, `status`
`sudo reboot`	Restart the Raspberry Pi	Persistence testing	Requires reconnecting over SSH afterward
`sudo poweroff`	Shutdown the Raspberry Pi safely	Physical WAN cutover	Wait for shutdown before removing power

A.2 Network Inspection and Interface Management

Command	Purpose	Typical Use	Notes
`ip link show`	Display network interfaces and link states	Interface identification	Shows states such as `UP`, `DOWN`, `NO-CARRIER`
`ip -br link`	Brief interface/link display	Quick interface check	Brief version of `ip link show`

`ip -4 -br address`	Display IPv4 addresses by interface	LAN/WAN verification	`-4` limits output to IPv4 and `-br` gives brief output
`ip -4 -br address show <interface>`	Display the IPv4 address of a specific interface	Verifying `eth0`, `eth1`, or `wlan0`	Replace `<interface>` with the appropriate interface name
`ip route`	Display the routing tables	Verify LAN/WAN routes	Default route should eventually use `eth0`
`lsusb`	List detected USB hardware	USB Ethernet adapter setup	Used to confirm the adapter
`nmcli device status`	Show NetworkManager interface states	LAN/WAN verification	Displays interface, type, state, and active profile
`nmcli connection show`	Show NetworkManager profiles	WAN cutover/configuration	Useful for identifying existing profiles
`nmcli connection up <profile>`	Activate a NetworkManager profile	LAN setup	Example: `pi-lan`
`nmcli connection down <profile>`	Deactivate a profile	Disable old Wi-Fi uplink	Used during final cutover
`nmcli -g connection.autoconnect connection show <profile>`	Displays the autoconnect setting of a profile	Persistence validation	Returns whether the profile automatically activates during boot

A.3 DHCP and DNS

Command	Purpose	Typical Use	Notes
`sudo dnsmasq --test`	Check dnsmasq configuration syntax	DHCP/DNS setup	Should report syntax OK

`sudo systemctl restart dnsmasq`	Restarts the DHCP/DNS service	Applying configuration changes	Existing clients may retain their current DHCP leases
`systemctl is-active dnsmasq`	Reports whether dnsmasq is currently running	Service validation	Expected result is `active`
`systemctl is-enabled dnsmasq`	Reports whether dnsmasq starts automatically at boot	Persistence validation	Expected result is `enabled`
`sudo journalctl -u dnsmasq -f`	Follow dnsmasq service logs live	DHCP troubleshooting	Stop display with `Ctrl+C`
`ss -luntp`	Show listening network sockets	DHCP/DNS verification	Used to confirm ports 53 and 67
`nslookup example.com 192.168.50.1`	Sends a DNS query directly to the Raspberry Pi	DNS Validation	Confirms that the Pi can resolve domain names for LAN client

A.4 Routing, NAT, and Firewall

Command	Purpose	Typical Use	Notes
`sysctl -w net.ipv4.ip_forward=1`	Temporarily enable IPv4 forwarding	Initial routing test	Resets after reboot unless made persistent
`sudo sysctl --system`	Reloads system `sysctl` configuration files	Applying persistent forwarding configuration	Used after creating `/etc/sysctl.d/99-pi-router.conf`
`sudo nft list ruleset`	Display all `nftables` rules	Firewall inspection	Useful before adding custom rules
`sudo nft list table ip pi_router`	Display the project's `nftables` table	Routing/firewall verification	Shows counter and active rules
`sudo nft --check --file <file>`	Validate `nftables` file syntax	Before firewall reload	Does not apply rules
`sudo systemctl restart nftables`	Reloads the saved `nftables` configuration	Applying persistent firewall rules	Loads `/etc/nftables.conf`

`systemctl is-active nftables`	Reports whether the `nftables` service is running	Firewall verification	Expected result is `active`
`systemctl is-enabled nftables`	Reports whether the `nftables` service starts automatically at boot	Persistence verification	Expected result is `enabled`
`watch -n 1 '<command>'`	Repeat a command periodically	Live monitoring	Useful for `nftables` counters

A.5 File and Configuration Utilities

Command	Purpose	Typical Use	Notes
`sudo nano <file>`	Edit a text file	`dnsmasq` configuration	Save with `Ctrl+O`; exit with `Ctrl+X`
`cp`	Copy a file	Firewall backup	Used to create rollback copy
`grep`	Search text for matching patterns	Firewall inspection	Often used with options such as `-nE`
`sed -i`	Edit matching text directly in a file	WAN cutover	Creates immediate in-place changes; a backup should be made first
`sudo tee <file>`	Writes command input into a file with administrator privileges	Creating configuration files	Used with here-documents for `dnsmasq` and `sysctl` configuration

A.6 Remote Access and Basic Connectivity

Command	Purpose	Typical Use	Notes
`ping <hostname>.local`	Sends ICMP Echo Request packets to a hostname	Verifying that the Raspberry Pi is reachable before SSH	Successful replies confirm local connectivity

			and hostname resolution
`ssh <username>@<hostname>.local`	Opens a Secure Shell session using the Pi's hostname	Initial remote administration	Requires SSH to be enabled
`ssh <username>@192.168.50.1`	Opens a Secure Shell session through the Pi's LAN address	Administration after LAN configuration	Important before disconnecting the temporary Wi-Fi uplink
`ping 192.168.50.1`	Tests connectivity to the Raspberry Pi LAN interface	LAN validation	Confirms that the client can reach the router
`ping 1.1.1.1`	Test Internet connectivity without relying on DNS	Routing/NAT validation	Useful for separating routing problems from DNS problems
`ping example.com`	Tests DNS resolution and Internet connectivity together	End-to-end validation	Requires DNS to function before the ping can begin

Appendix B – Client Network Validation

Test	Command	Confirms
Network configuration	`ipconfig /all`	DHCP address, subnet, gateway, DNS
LAN connectivity	`ping 192.168.50.1`	Client can reach Pi LAN interface
Internet routing	`ping 1.1.1.1`	Routing/NAT works without depending on DNS
DNS	`nslookup example.com 192.168.50.1`	Pi DNS service works
HTTPS	`curl.exe -I https://example.com`	Normal application traffic reaches Internet

Route path	`tracert 1.1.1.1`	Shows path/hops toward Internet